

Índice

1	El problema	1
2	Implementación	2
2.1	Formato de la entrada	2
2.2	Formato de la salida	2
3	Ficheros de partida	2
3.1	Ficheros de prueba (public.zip)	3
3.2	Fichero principal (e4.py)	3
4	Entrega en el aula virtual	4
5	Calificación del entregable	5
5.1	Medición de tiempos de ejecución	6
5.2	Errores graves en el entregable	6
5.3	Penalización por copia	6

1. El problema

Una biblioteca ha recibido una colección de CDs de una donación. Los ha recibido en un tubo de cartón donde están apilados unos encima de otros, sin sus cajas y sin orientación (unos están boca arriba y otros boca abajo). La biblioteca dispone de un brazo robot que puede coger la cantidad deseada de CDs del tope de la pila, darles la vuelta y volver a colocarlos en la pila. Lógicamente, en la biblioteca quieren dejar los CDs de modo que todos tengan la misma orientación (con el lado de la etiqueta hacia arriba). Y también quieren minimizar el número de operaciones que debe hacer el robot.

Deberéis implementar un programa que, dada una descripción de la pila de CDs, encuentre la secuencia de operaciones del brazo robot de mínima longitud que deje todos los CDs con la etiqueta hacia arriba. El coste temporal del algoritmo deberá ser $\mathcal{O}(n)$, donde n es el número de CDs.

Más formalmente, la pila de CDs será una lista P de letras que representan la orientación del CD: “u” si tiene la etiqueta arriba y “d” si tiene la etiqueta abajo. El programa deberá generar una lista de enteros que representan las operaciones del brazo. Cuando se aplica el entero p a la pila P se obtiene una nueva pila en la que se invierte el orden de los p primeros elementos de P y se cambia su orientación. Por ejemplo, si P es ['u', 'd', 'd', 'u', 'd'] y se aplica el entero 3, sucede lo siguiente:

- Se cogen los tres primeros CDs: ['u', 'd', 'd'].
- Se invierte su orden: ['d', 'd', 'u'].
- Se cambia su orientación: ['u', 'u', 'd'].
- Se vuelven a colocar en la pila que ahora es ['u', 'u', 'd', 'u', 'd'].

Observa que el tope de la pila es el primer elemento de la lista.

Siguiendo con el ejemplo anterior, una secuencia que orienta los CDs correctamente es la siguiente:

Operación	Pila
—	['u', 'd', 'd', 'u', 'd']
4	['d', 'u', 'u', 'd', 'd']
1	['u', 'u', 'u', 'd', 'd']
3	['d', 'd', 'd', 'd', 'd']
5	['u', 'u', 'u', 'u', 'u']

Hay más soluciones posibles. Por ejemplo, podemos ordenar la pila con las operaciones 1, 3, 4 y 5. En cualquier caso, el número mínimo de operaciones es 4.

2. Implementación

Tenéis que implementar un programa de línea de órdenes, `e4.py`, que lea la instancia desde la entrada estándar y muestre el resultado por la salida estándar. Los formatos de entrada y salida se especifican en los dos apartados siguientes.

Para poder optar a la máxima nota, el coste temporal del programa deberá ser $\mathcal{O}(n)$, donde n es el número de CDs. Para ello, deberéis utilizar una estrategia voraz y estructuras de datos adecuadas.

2.1. Formato de la entrada

Una instancia es un archivo de texto que contiene una cadena con una letra por CD. La letra es `u` si el CD tiene la etiqueta hacia arriba y `d` si la tiene hacia abajo. Por ejemplo, la instancia de la introducción es:

```
uddud
```

El orden de los CDs en esta cadena es el mismo que en la lista, es decir, el tope de la pila se corresponde con el primer carácter de la cadena.

2.2. Formato de la salida

La salida del programa se mostrará por la salida estándar y presentará los enteros correspondientes a las operaciones del brazo robot, uno por línea. Por ejemplo, la salida del ejemplo de la introducción sería:

```
4
1
3
5
```

En caso de haber más de una solución, se aceptará cualquiera de ellas.

3. Ficheros de partida

En el aula virtual disponéis de una carpeta con el siguiente contenido:

- `public.zip`: un archivo comprimido que contiene la carpeta `public` con un conjunto de instancias de prueba y sus soluciones.

- `e4.py`: el programa que debéis de utilizar como punto de partida y que hay que entregar.

Los siguientes apartados explican con detalle estos ficheros.

3.1. Ficheros de prueba (`public.zip`)

Al descomprimir el fichero `public.zip` obtendréis la carpeta `public`, que contiene las instancias de prueba y sus soluciones, los ficheros `e4-*.i` y `e4-*.o`, respectivamente. El nombre de las instancias sigue el patrón `e4-<num_ins>-<tamaño>-<instrucciones>.i`, donde:

- `<num_ins>`: el número de instancia, un número entre 1 y 10.
- `<tamaño>`: el tamaño de la instancia, esto es, el número de días.
- `<instrucciones>`: el número mínimo de instrucciones de la instancia.

3.2. Fichero principal (`e4.py`)

Contiene la estructura del programa. Debéis modificarlo teniendo en cuenta lo siguiente:

- Podéis añadir funciones o clases adicionales, pero no debéis modificar nada del programa principal. Modificar el programa principal supondrá un cero en la calificación del entregable. Tampoco podéis modificar la firma (el tipo de los parámetros y el tipo devuelto) de las tres funciones que se utilizan en el programa principal.
- Podéis importar módulos de la biblioteca estándar de Python, pero la única biblioteca externa permitida es `algorithmia`. Por ejemplo, `NumPy` es una biblioteca externa y, por lo tanto, no está permitida.

Veamos la estructura de `e4.py` y las funciones que debéis implementar.

El tipo `Data` simplemente es una lista de booleanos:

```
type Data = list[bool]
```

El valor `True` indica que el CD correspondiente tiene la etiqueta hacia arriba; `Falso`, que la tiene hacia abajo.

Y el tipo `Result` es una lista de enteros:

```
type Result = list[int]
```

Cada entero representa una operación del brazo robot.

El programa principal utiliza la estructura de tres funciones vista en clase:

```
if __name__ == "__main__":  
    data0 = read_data(sys.stdin)  
    result0 = process(data0)  
    show_result(result0)
```

Por lo tanto, el programa contiene las funciones `read_data`, `process` y `show_result`:

- `read_data`: recibe `f`, un fichero (no su nombre), y devuelve la lista de CDs. Su firma es:

```
def read_data(f: TextIO) -> Data:
```
- `process`: recibe el data leído por `read_data` y devuelve las órdenes para el brazo robot. Su firma es:

```
def process(data: Data) -> Result:
```

- `show_result`: recibe `result` y lo muestra por la salida estándar. Su firma es:

```
def show_result(result: Result):
```

La función `show_result` ya está implementada.

Cómo ejecutar vuestro programa `e4.py`

Dado que el programa lee de la entrada estándar, no resulta cómodo lanzarlo desde PyCharm. Se recomienda ejecutarlo desde el terminal que incorpora el propio PyCharm (el botón está en la parte inferior izquierda). Una vez abierto, utilizad la orden `cd` para ir al directorio donde estén `e4.py` y `public`.

Una vez en el directorio, podéis ejecutar el programa con la siguiente orden:

- Linux y MacOS: `python3 e4.py < public/e4-01-1000-1854.i`
- Windows (CMD): `python e4.py < public\e4-01-1000-1854.i`
- Windows (PowerShell): `GetContent public\e4-01-1000-1854.i | python e4.py`

Cómo comprobar la salida

Cuando comprobéis la salida de vuestro programa, lo primero que tenéis que mirar es si coincide vuestro número de órdenes con el indicado en el nombre del fichero de prueba. Como es posible que no coincida, necesitaréis un diagnóstico más detallado. Por eso os aconsejamos que empecéis por escribir un programa que verifique que vuestro `process` funciona correctamente. Para ello, podéis aprovechar la estructura de `e4.py` e importar `read_data` y `process`. El programa principal puede ser:

```
if __name__ == "__main__":
    t0 = time.process_time()
    data0 = read_data(sys.stdin)
    result0 = process(data0[:])
    t1 = time.process_time()
    print(f"Total time: {t1 - t0:.3f} seconds")

    check = correct_solution(data0, result0)
    if check is True:
        print("OK")
    else:
        print(f"WRONG SOLUTION: {check}")
```

La primera parte ejecuta `process` y mide el tiempo de ejecución. Esto os dará una estimación, aunque debéis utilizar `rei` para medir el tiempo que se utilizará para la calificación, como se explica en la sección 5.1. Después añadid las comprobaciones que consideréis oportunas. En este ejemplo se ejecuta `correct_solution`, que devuelve `True` si la solución es correcta y el estado final de la pila de CDs, si no lo es. Observa que pasamos una copia de `data0` a `process` por si `process` modifica su parámetro. También tenemos que comparar `check` con `True` porque `if check:` considera una lista no vacía como cierto.

4. Entrega en el aula virtual

La entrega consistirá en subir dos ficheros a la tarea correspondiente del aula virtual, solo debe subirlos uno de los miembros de la pareja. Estos son los dos ficheros:

- `e4.py`: El fichero con el código del entregable.
- El fichero con la identificación del grupo. Para parejas, el fichero se llamará `alxxxxxx_alyyyyyy.txt` y contendrá dos líneas, cada una con el nombre completo de un miembro de la pareja. En las entregas individuales, el fichero se llamará `alxxxxxx.txt` y contendrá una línea con el nombre completo de la persona que realiza la entrega. En ambos casos, `alxxxxxx` y `alyyyyyy` son los `al` de los miembros de la pareja. Por ejemplo, si la pareja está formada por Elena Nito (`al000000`) y Aitor Menta (`al111111`), el fichero se llamará `al000000_al111111.txt` y su contenido será:

```
Elena Nito
Aitor Menta
```

Si Elena entrega individualmente, el fichero se llamará `al000000.txt` y su contenido será:

```
Elena Nito
```

Vuestra entrega debe cumplir las siguientes restricciones:

- Solo debéis subir los dos ficheros solicitados.
- Los nombres de los dos ficheros deben utilizar únicamente minúsculas.
- El carácter “`_`” que aparece en `alxxxxxx_alyyyyyy.txt` es el guion bajo, no utilizéis el carácter “`-`” ni ningún otro carácter similar como separador.

Información importante si utilizáis Windows para subir los ficheros al aula virtual

Si usáis Windows, es probable que el administrador de archivos os oculte algunas extensiones de archivo, pues, incomprensiblemente, es la configuración por defecto de Windows. En ese caso, es posible que subáis los archivos con doble extensión (`e4.py.py` y `alxxxxxx_alyyyyyy.txt.txt`) porque la extensión que tiene el archivo no la veréis y le añadiréis otra. Como consecuencia, los nombres de los archivos no serán los correctos y se penalizará la nota con un punto. Para evitar este error, configurad el administrador de archivos de Windows para que muestre las extensiones de todos los archivos.

5. Calificación del entregable

El entregable se calificará sobre 10 utilizando nuevas instancias, de tamaño similar al de las instancias públicas. Las nuevas instancias se publicarán junto con las calificaciones para que podáis probarlas con vuestro programa.

Una instancia se considerará superada si se cumplen estas dos condiciones:

- La solución obtenida es óptima.
- El tiempo acumulado de las funciones `read_data` y `process` no ha superado el tiempo máximo de ejecución: un segundo.

Con los problemas más pequeños es posible obtener la solución óptima incluso con un algoritmo muy ineficiente, con los medianos se necesita un algoritmo más eficiente. Con los problemas más grandes necesitaréis un algoritmo con un coste $\mathcal{O}(n)$, siendo n el número de CDs.

Importante: podéis detectar y corregir muchos errores en vuestro programa utilizando las instancias públicas, aunque superar las instancias públicas no garantiza superar las privadas, sobre todo si la implementación se ha “ajustado” específicamente para superar las públicas.

5.1. Medición de tiempos de ejecución

Para poder medir tiempos de forma consistente necesitamos un ordenador de referencia. Para este entregable será `rei.dlsi.uji.es`.

Para obtener el tiempo de ejecución sobre el ordenador de referencia, podéis subir vuestro programa a

<https://rei.dlsi.uji.es/algoritmia>

donde un servicio lo ejecutará para todas las instancias y mostrará el tiempo de ejecución de las funciones `read_data` y `process` para cada una de ellas.

Algunas consideraciones sobre la página web:

- Para conectaros al servidor desde fuera de la UJI, necesitaréis utilizar la VPN de la universidad. Si no sabes cómo configurarla, podéis encontrar toda la información en

<https://connectaxarxa.uji.es/es/index.html#vpn>

Tened en cuenta que al activar la VPN recibiréis un correo con un enlace que debéis pulsar para que la VPN no se cierre automáticamente tras unos pocos minutos (es el doble factor).

- El servidor `rei.dlsi.uji.es` no tiene IP pública por lo que su certificado es autofirmado. Tendréis que aceptar una excepción de seguridad para poder acceder a la página.
- El servicio sólo muestra el tiempo de ejecución de vuestra función `process`, no comprueba que la solución que obtiene vuestro programa sea correcta, eso debéis comprobarlo vosotros.

5.2. Errores graves en el entregable

Se calificarán con un cero aquellos entregables que hayan modificado el programa principal de `e4.py` o los tipos de cualquiera de las funciones que utiliza.

También se penalizará que el contenido de la salida estándar no respete el formato que se especifica en el apartado 2.2. Una salida errónea puede tener dos causas:

- Una modificación de `show_result` que haga que no se respete el formato especificado. Se penalizará la nota con dos puntos.
- Algún `print` de depuración olvidado en el código. Se penalizará la nota con un punto. Para que no os pase, podéis utilizar la salida de error estándar, por ejemplo:

```
print("Este mensaje se mostrará en la salida de error estándar", file=sys.stderr)
```

Por último, también se penalizará con un punto cada restricción incumplida en la entrega al aula virtual (ver el apartado 4). Revisad la entrega con detenimiento antes de subirla al aula virtual (los dos miembros de la pareja).

5.3. Penalización por copia

En caso de detectarse una copia, se aplicará la normativa de la universidad: la calificación de la evaluación continua (60 % de la nota final) será de cero en la primera convocatoria para todos los involucrados.